YACHAY TECH UNIVERSITY

//01

Graph Traversal Algorithms

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Graph Traversal Algorithms

Many problems require processing all graph nodes (and edges) in a systematic way.

Graph traversal algorithms:

- Depth-first search (DFS)
- Breadth-first search (BFS)

Depth-First Search (DFS)

- Visits graph nodes by always moving away from the last visited node to the unvisited one. It backtracks if no adjacent unvisited nodes are available.

- Uses a stack
  - a node is pushed onto the stack when it's reached for the first time.
  - a node is popped off the stack when it becomes a dead-end, i.e., when there are no adjacent unvisited nodes.

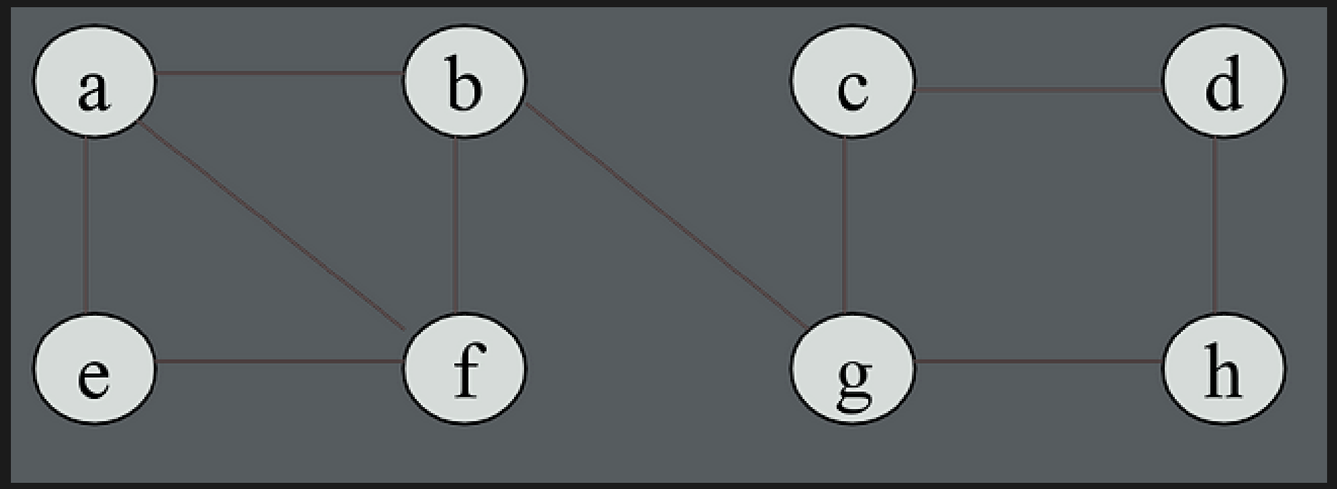
- "Redraws" graph in tree-like fashion (with tree edges and back edges for undirected graph)

Pseudocode of DFS

```
ALGORITHM  DFS(G)
    //Implements a depth-first search traversal of a given graph
    //Input: Graph G = ⟨V, E⟩
    //Output: Graph G with its vertices marked with consecutive integers
    //in the order they've been first encountered by the DFS traversal
    mark each vertex in V with 0 as a mark of being "unvisited"
    count ← 0
    for each vertex v in V do
        if v is marked with 0
            dfs(v)

    dfs(v)
    //visits recursively all the unvisited vertices connected to vertex v by a path
    //and numbers them in the order they are encountered
    //via global variable count
    count ← count + 1; mark v with count
    for each vertex w in V adjacent to v do
        if w is marked with 0
            dfs(w)
```

Example: DFS traversal of undirected graph



DFS traversal stack:

DFS tree:

Notes on DFS

- DFS can be implemented with graphs represented as:
  - adjacency matrices: $\Theta(V^2)$
  - adjacency lists: $\Theta(|V|+|E|)$

- Yields two distinct ordering of nodes:
  - order in which nodes are first encountered (pushed onto the stack)
  - order in which nodes become dead-ends (popped off the stack)

- Applications:
  - checking connectivity, finding connected components
  - checking acyclicity
  - finding articulation points and biconnected components
  - searching state-space of problems for solution (AI)

Breadth-first search (BFS)

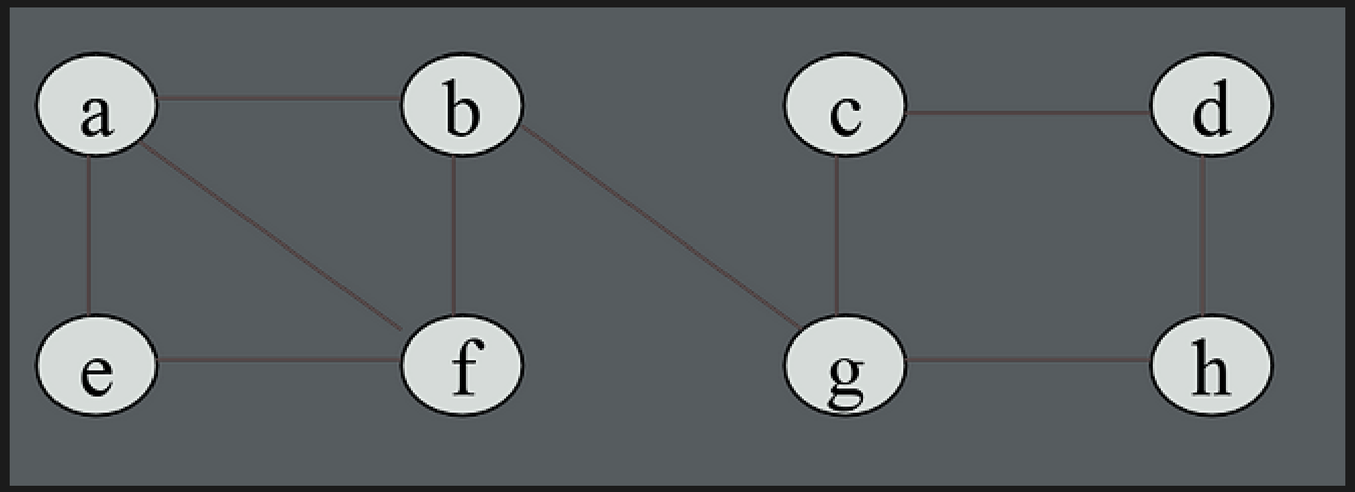
- Visits graph nodes by moving across to all the neighbors of the last visited node
- Instead of a stack, BFS uses a queue
- Similar to level-by-level tree traversal
- “Redraws” graph in tree-like fashion (with tree edges and cross edges for undirected graph)

Pseudocode of BFS

```
ALGORITHM  BFS(G)
    //Implements a breadth-first search traversal of a given graph
    //Input: Graph G = ⟨V, E⟩
    //Output: Graph G with its vertices marked with consecutive integers
    //in the order they have been visited by the BFS traversal
    mark each vertex in V with 0 as a mark of being "unvisited"
    count ← 0
    for each vertex v in V do
        if v is marked with 0
            bfs(v)

    bfs(v)
    //visits all the unvisited vertices connected to vertex v by a path
    //and assigns them the numbers in the order they are visited
    //via global variable count
    count ← count + 1;   mark v with count and initialize a queue with v
    while the queue is not empty do
        for each vertex w in V adjacent to the front vertex do
            if w is marked with 0
                count ← count + 1;   mark w with count
                add w to the queue
        remove the front vertex from the queue
```

Example of BFS traversal of undirected graph



BFS traversal queue:

BFS tree:

Notes on BFS

- BFS has the same efficiency as DFS and can be implemented with graphs represented as:
  - adjacency matrices: $\Theta(V^2)$
  - adjacency lists: $\Theta(|V|+|E|)$
- Yields single ordering of nodes (order added/deleted from the queue is the same)
- Applications: same as DFS, but can also find paths from a node to all other nodes with the smallest number of edges

Closest Pair and Convex Hull

Israel Pineda, Ph.D.
School of Mathematical and Computational Sciences

Closest-Pair Problem

Find the two closest points in a set of n points (in the two-dimensional Cartesian plane).

Brute-force algorithm

Compute the distance between every pair of distinct points and return the indexes of the points for which the distance is the smallest.

Closest-Pair Brute-Force Algorithm (cont.)

ALGORITHM *BruteForceClosestPoints*(P)

```
//Input: A list P of n (n ≥ 2) points P_1 = (x_1, y_1), ..., P_n = (x_n, y_n)
//Output: Indices index1 and index2 of the closest pair of points
dmin ← ∞
for i ← 1 to n − 1 do
    for j ← i + 1 to n do
        d ← sqrt((x_i − x_j)^2 + (y_i − y_j)^2) //sqrt is the square root function
        if d < dmin
            dmin ← d; index1 ← i; index2 ← j
return index1, index2
```

Efficiency:

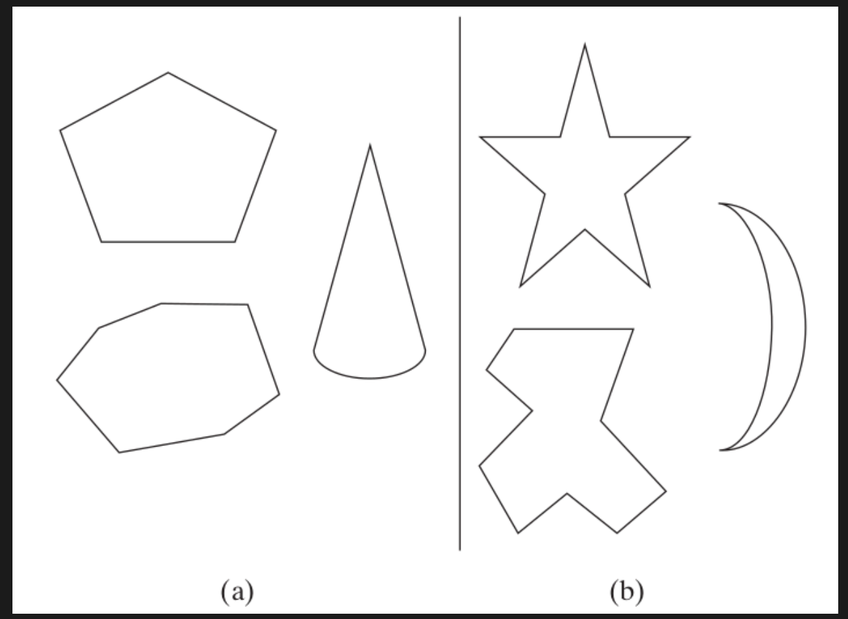
How to make it faster?

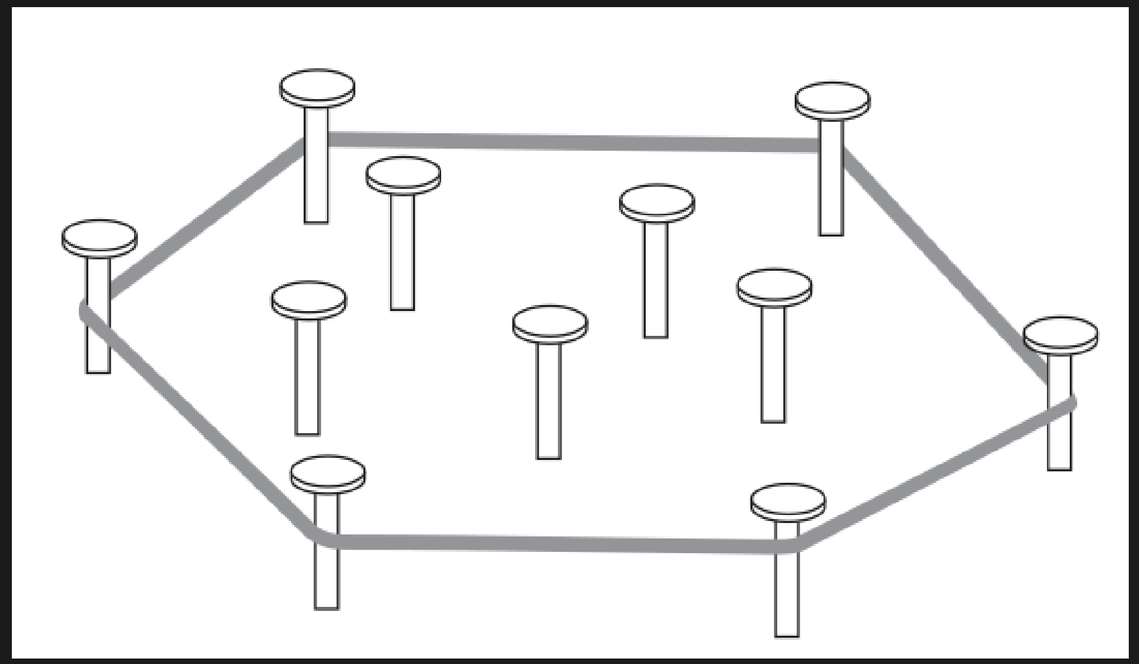
Convex Hull

DEFINITION A set of points (finite or infinite) in the plane is called **convex** if for any two points p and q in the set, the entire line segment with the endpoints at p and q belongs to the set.

DEFINITION The **convex hull** of a set S of points is the smallest convex set containing S. (The “smallest” requirement means that the convex hull of S must be a subset of any convex set containing S.)

Convex Hull



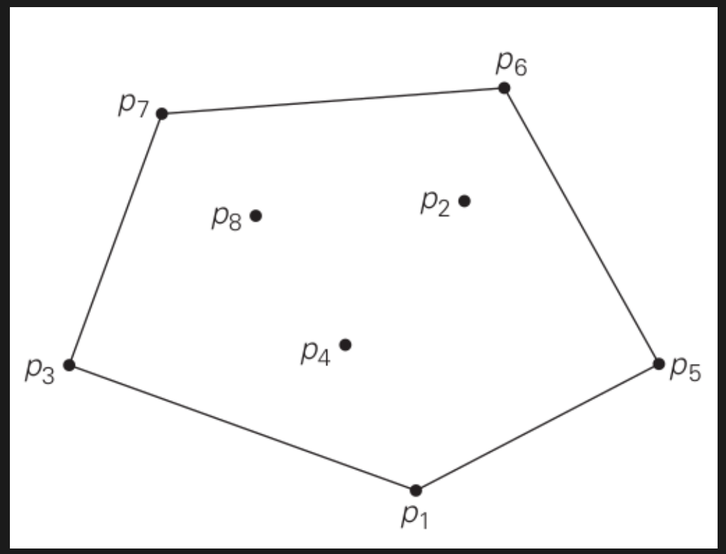


a) Convex sets. b) Sets that are not convex.

Rubber-band interpretation of the convex hull.

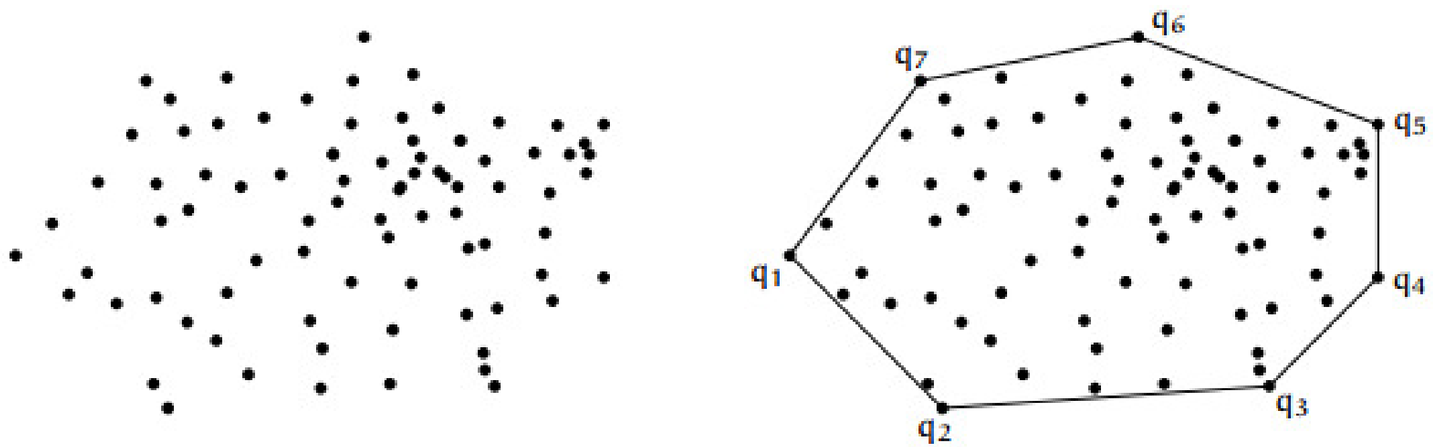
Convex Hull

THEOREM The convex hull of any set S of n > 2 points not all on the same line is a convex polygon with the vertices at some of the points of S. (If all the points do lie on the same line, the polygon degenerates to a line segment but still with the endpoints at two points of S.)



The convex hull for this set of eight points is the convex polygon with vertices p_1, p_5, p_6, p_7, and p_3

Convex Hull Problem



(a) Input. (b) Output.

Convex Hull Algorithm

- A line segment connecting two points P_i and P_j of a set of n points is part of its convex hull's boundary if and only if all the other points of the set lie on the same side of the straight line through these two points.

- Repeating this test for every pair of points yields a list of line segments that make up the convex hull's boundary.

Convex Hull Algorithm

```
BruteForce_ConvexHull(P){
  E = empty set
  for all ordered pairs (p,q) in PxP, and p!=q do {
    isEdge = true
    for all points r in P, r!= p and r!=q do {
      if r lies to the left of directed line from p to q
        isEdge = false
    }
    if isEdge then
      add pq to E
  }

  construct a list L of vertices in CW order from E
  return L
}
```

Convex Hull Algorithm

- Computation time fo convex hull using brute force algorithm is $O(n^3)$
  - $O(n)$ complexity tests, for each of $O(n^2)$ edges

- The time efficiency of this algorithm is in $O(n^3)$.
  - For each of $n(n-1)/2$ pairs of distinct points, we may need to find the sign of $ax + by - c$ for each of the other $n-2$ points.

Q & A

FOR QUESTIONS OR COMMENTS

EMAIL ADDRESS

ipineda@yachaytech.edu.ec